Angular HandsOn 10: Unit Testing with Jasmine and Karma

HANDS-ON 10 [Advanced]

Unit Testing Angular Applications ? Jasmine, Karma & TestBed

Topics Covered:

Jasmine Syntax ? describe, it, expect, [Check]

spyOn

TestBed for Angular Testing[Check]

Component Testing ? fixture, [Check]

debugElement

Testing @Input and @Output[Check]

Service Testing with [Check]

HttpClientTestingModule

Component + Store Testing with MockStore[Check]

Every spec.ts file Angular generated has been waiting ? now you will actually write tests. This hands-on covers unit testing Angular components and services using Jasmine, Karma, and Angular's TestBed utilities.

RUN ng test ? runs all spec.ts files in watch mode using Karma. Press Ctrl+C t- stop. Run ng test --code-coverage t- generate a coverage report in coverage/ folder.

Task 1: Testing a Component ? CourseCardComponent

Goal: Write unit tests for CourseCardComponent covering rendering, inputs, and output events.

Steps:

101. Open course-card.component.spec.ts. Inside the existing describe block, add a beforeEach that configures TestBed: TestBed.configureTestingModule({ declarations: [CourseCardComponent] }) (or imports for standalone). Call fixture = TestBed.createComponent(CourseCardComponent); component = fixture.componentInstance;.

102. Write a test that verifies the component is created: it('should create', () => { expect(component).toBeTruthy(); });.

103. Write a test for @Input rendering: set component.course = { id:1, name:'Data Structures', code:'CS101', credits:4, gradeStatus:'passed' }; fixture.detectChanges(); then use fixture.debugElement.query(By.css('h3')).nativeElement.textContent t- assert the course name is displayed.

104. Write a test for @Output: set component.course = mockCourse; fixture.detectChanges(); spyOn(component.enrollRequested, 'emit'); fixture.debugElement.query(By.css('button')).nativeElement.click(); fixture.detectChanges(); expect(component.enrollRequested.emit).toHaveBeenCalledWith(1);.

105. Write a test for ngOnChanges: call component.ngOnChanges with a SimpleChanges object and verify the expected console.log was called (spy on console.log).

Hint:
- fixture.detectChanges() triggers Angular's change detection ? always call it after changing component properties before querying the DOM.
- By.css() (from @angular/platform-browser') is the Angular way t- query the rendered DOM in tests ? use it instead of document.querySelector t- stay within the component's scope.
Expected Outcome: ng test shows 5 passing tests for CourseCardComponent. Coverage report shows the component's TypeScript logic is covered.

Task 2: Testing a Service and an NgRx-Connected Component

Goal: Test CourseService with HttpClientTestingModule and test an NgRx store-connected component.
Steps:
106. Open course.service.spec.ts. Configure TestBed with HttpClientTestingModule: TestBed.configureTestingModule({ imports: [HttpClientTestingModule], providers: [CourseService] }). Inject both the service and HttpTestingController.

107. Write a test for getCourses(): call service.getCourses().subscribe(courses => expect(courses.length).toBe(2)); then use httpMock.expectOne('http://localhost:3000/courses').flush(mockCourses); and httpMock.verify() t- assert n- unexpected requests were made.
108. Write a test for error handling: flush a 500 error response and assert the Observable emits an error with the expected message.
109. Test a component that uses the NgRx store: configure TestBed with provideMockStore({ initialState: { course: { courses: mockCourses, loading: false, error: null } } }). Inject MockStore. Call fixture.detectChanges() and assert the rendered course cards match the initial state.
110. Use store.setState({ course: { courses: [], loading: true, error: null } }) t- simulate a loading state. Call fixture.detectChanges() and assert the loading indicator is visible in the DOM.
Hint:
- HttpTestingController.verify() asserts that there are n- outstanding (unsatisfied) HTTP requests after each test ? critical for catching tests that make unintended HTTP calls.
- MockStore from @ngrx/store/testing replaces the real store with a controllable mock ? you can set the state and dispatch actions without running real reducers or effects.
Expected Outcome: ng test shows all service tests passing. The getCourses test verifies the correct URL was

called. The MockStore component test shows loading indicator when loading is true.